

Python Essential 1 Course By Cisco Networking Academy

PYTHON FINAL PROJECT

Mini Cyber Security Toolkit (Python CLI Based Tool)

CODE LINK: <https://github.com/zunairaarshadgit/Mini-Cyber-Security-Toolkit-Python-CLI-Based-Tool->

INTRODUCTION:

This project is a command-line based cybersecurity toolkit developed in Python. It provides multiple security-related utilities such as URL analysis, password strength checking, port scanning, file hashing, and a basic login system. The goal of this project is to implement fundamental cybersecurity concepts practically and enhance problem-solving skills.

OBJECTIVES:

- Learn Python practically
- Understand cybersecurity basics
- Develop CLI-based tools
- Implement real-world logic (validation, scanning, hashing)

FEATURES:

- ◆ URL Checker
- ◆ Password Checker
- ◆ Port Scanner
- ◆ File Hasher
- ◆ Login System

TECHNOLOGIES USED:

- Python
- socket module (networking)
- hashlib (encryption/hashing)
- re (regex validation)

5. WORKING / LOGIC

The project is based on a command-line interface (CLI) menu system that continuously runs using a while True loop. This loop ensures that the program does not terminate after a single execution and allows the user to perform multiple operations.

When the program starts, a menu is displayed with different options such as URL Checker, Password Checker, Port Scanner, File Hasher, and Login System. The user selects an option by entering a corresponding number.

The program uses a match-case control structure to handle user input. Based on the selected option, the relevant function is called. Each module performs its specific task and displays the result on the screen.

The general flow of the program is as follows:

- The user selects an option from the menu
- The corresponding function is executed
- The result is displayed
- The program returns to the main menu for further operations

This loop continues until the user selects the exit option.

6. SECURITY CONCEPTS USED

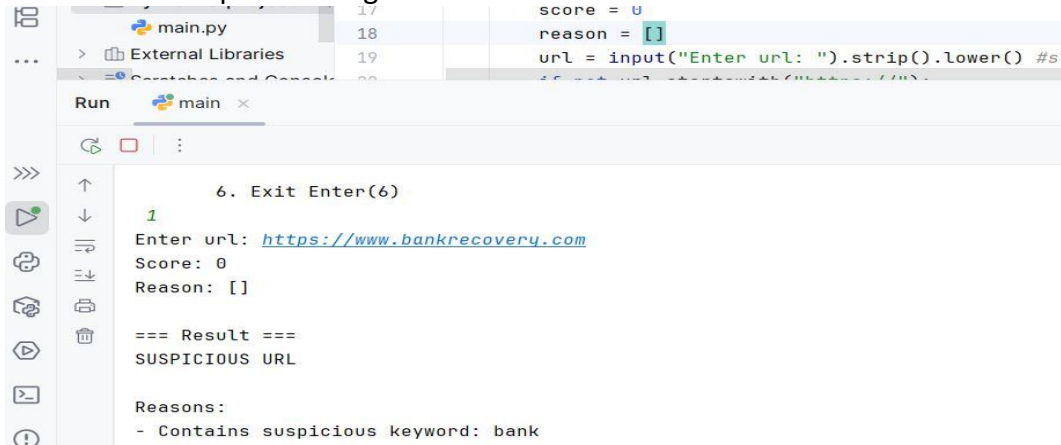
This project implements several fundamental cybersecurity concepts:

- **Basic Phishing Detection:**
The URL Checker analyzes URLs for suspicious patterns such as missing HTTPS, unusual length, excessive dots, and the presence of suspicious keywords.
- **Password Strength Analysis:**
The Password Checker evaluates passwords based on multiple criteria including length, uppercase/lowercase letters, digits, and special characters to determine whether a password is weak or strong.
- **Port Scanning (Network Security):**
The Port Scanner checks open ports on a given IP address using socket programming and multithreading. This demonstrates how network vulnerabilities can be identified.
- **Hashing (Data Protection):**
The File Hasher uses SHA-256 hashing to convert input data into a secure, fixed-length hash, ensuring data integrity and confidentiality.

7. OUTPUT / SCREENSHOTS

The project generates outputs for each module, which can be captured as screenshots for documentation purposes:

- URL Checker output showing risk level and reasons



```
17 score = 0
18 reason = []
19 url = input("Enter url: ").strip().lower()
20 if not url.startswith("https://"):
21     score -= 1
22     reason.append("Missing https://")
23 if len(url) > 60:
24     score -= 1
25     reason.append("Unusually long URL")
26 if "." in url:
27     score -= 1
28     reason.append("Excessive dots")
29 suspicious_keywords = ["bank", "credit", "loan", "investment", "phishing"]
30 for keyword in suspicious_keywords:
31     if keyword in url:
32         score -= 1
33         reason.append(f"Contains suspicious keyword: {keyword}")
34
35 def url_checker(url):
36     score = 0
37     reason = []
38     url = url.strip().lower()
39     if not url.startswith("https://"):
40         score -= 1
41         reason.append("Missing https://")
42     if len(url) > 60:
43         score -= 1
44         reason.append("Unusually long URL")
45     if "." in url:
46         score -= 1
47         reason.append("Excessive dots")
48     suspicious_keywords = ["bank", "credit", "loan", "investment", "phishing"]
49     for keyword in suspicious_keywords:
50         if keyword in url:
51             score -= 1
52             reason.append(f"Contains suspicious keyword: {keyword}")
53
54     return score, reason
55
56 def main():
57     while True:
58         print("\n1. URL Checker Enter(1)")
59         url = input("Enter url: ")
60         score, reason = url_checker(url)
61         print(f"Score: {score}")
62         print(f"Reason: {reason}")
63
64         if score > 0:
65             print("SAFE URL")
66         else:
67             print("SUSPICIOUS URL")
68
69         print("\n2. Password Checker Enter(2)")
70         password = input("Enter password: ")
71         strength = password_checker(password)
72         print(f"Strength: {strength}")
73
74         print("\n3. Port Scanner Enter(3)")
75         ip = input("Enter IP address: ")
76         ports = input("Enter ports (comma-separated): ")
77         open_ports = port_scanner(ip, ports)
78         print(f"Open ports: {open_ports}")
79
80         print("\n4. File Hasher Enter(4)")
81         file_path = input("Enter file path: ")
82         hash_value = file_hasher(file_path)
83         print(f"Hash value: {hash_value}")
84
85         print("\n5. Login System Enter(5)")
86         username = input("Enter username: ")
87         password = input("Enter password: ")
88         login_status = login_system(username, password)
89         print(f"Login status: {login_status}")
90
91         print("\n6. Exit Enter(6)")
92         if input("Press 6 to exit: ") == "6":
93             break
94
95 if __name__ == "__main__":
96     main()
```

6. Exit Enter(6)

1

Enter url: <https://www.bankrecovery.com>

Score: 0

Reason: []

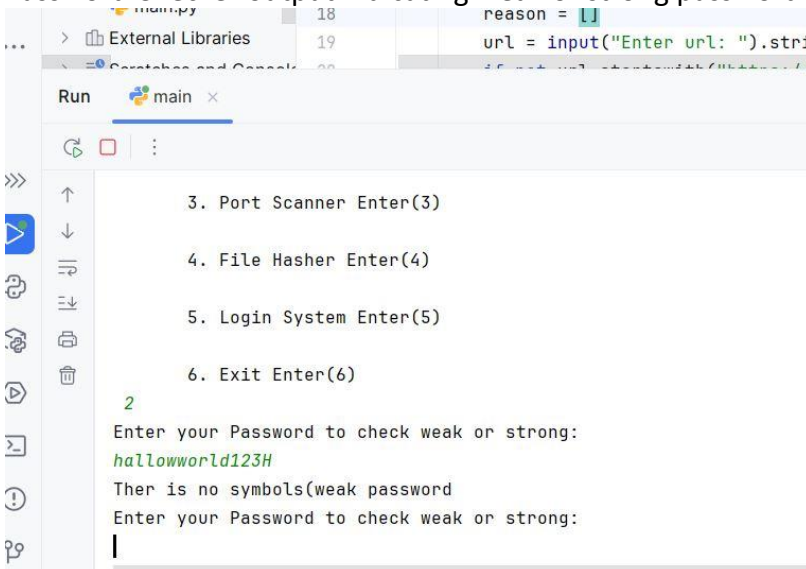
=== Result ===

SUSPICIOUS URL

Reasons:

- Contains suspicious keyword: bank

- Password Checker output indicating weak or strong password



```
17 score = 0
18 reason = []
19 url = input("Enter url: ").strip().lower()
20 if not url.startswith("https://"):
21     score -= 1
22     reason.append("Missing https://")
23 if len(url) > 60:
24     score -= 1
25     reason.append("Unusually long URL")
26 if "." in url:
27     score -= 1
28     reason.append("Excessive dots")
29 suspicious_keywords = ["bank", "credit", "loan", "investment", "phishing"]
30 for keyword in suspicious_keywords:
31     if keyword in url:
32         score -= 1
33         reason.append(f"Contains suspicious keyword: {keyword}")
34
35 def url_checker(url):
36     score = 0
37     reason = []
38     url = url.strip().lower()
39     if not url.startswith("https://"):
40         score -= 1
41         reason.append("Missing https://")
42     if len(url) > 60:
43         score -= 1
44         reason.append("Unusually long URL")
45     if "." in url:
46         score -= 1
47         reason.append("Excessive dots")
48     suspicious_keywords = ["bank", "credit", "loan", "investment", "phishing"]
49     for keyword in suspicious_keywords:
50         if keyword in url:
51             score -= 1
52             reason.append(f"Contains suspicious keyword: {keyword}")
53
54     return score, reason
55
56 def main():
57     while True:
58         print("\n1. URL Checker Enter(1)")
59         url = input("Enter url: ")
60         score, reason = url_checker(url)
61         print(f"Score: {score}")
62         print(f"Reason: {reason}")
63
64         if score > 0:
65             print("SAFE URL")
66         else:
67             print("SUSPICIOUS URL")
68
69         print("\n2. Password Checker Enter(2)")
70         password = input("Enter password: ")
71         strength = password_checker(password)
72         print(f"Strength: {strength}")
73
74         print("\n3. Port Scanner Enter(3)")
75         ip = input("Enter IP address: ")
76         ports = input("Enter ports (comma-separated): ")
77         open_ports = port_scanner(ip, ports)
78         print(f"Open ports: {open_ports}")
79
80         print("\n4. File Hasher Enter(4)")
81         file_path = input("Enter file path: ")
82         hash_value = file_hasher(file_path)
83         print(f"Hash value: {hash_value}")
84
85         print("\n5. Login System Enter(5)")
86         username = input("Enter username: ")
87         password = input("Enter password: ")
88         login_status = login_system(username, password)
89         print(f"Login status: {login_status}")
90
91         print("\n6. Exit Enter(6)")
92         if input("Press 6 to exit: ") == "6":
93             break
94
95 if __name__ == "__main__":
96     main()
```

3. Port Scanner Enter(3)

4. File Hasher Enter(4)

5. Login System Enter(5)

6. Exit Enter(6)

2

Enter your Password to check weak or strong:

[hallowworld123H](#)

Ther is no symbols(weak password)

Enter your Password to check weak or strong:

|

- Port Scanner output displaying open ports

```

>>>
4. File Hasher Enter(4)

5. Login System Enter(5)

6. Exit Enter(6)

3
Enter your IP Address: |
192.168.
[OPEN] Port 135
[OPEN] Port 139
[OPEN] Port 445
Mini Cyber Security Toolkit (Python CLI Project)

```

- File Hasher output showing the generated SHA-256 hash

```

Run main x
>>>
3. Port Scanner Enter(3)

4. File Hasher Enter(4)

5. Login System Enter(5)

6. Exit Enter(6)

4
Type any confidential message for convert to hash:
hey cherryberyy
Your message successfully convert into sha256: 8978be44bbc1cbd9421a285199e4ba4efac82e5aab1aa54d0e2e15d7664f2956
Mini Cyber Security Toolkit (Python CLI Project)

```

- Displays login success or failure, and blocks the user after three incorrect attempts to ensure basic security control.

When Wrong Passkey Enter:

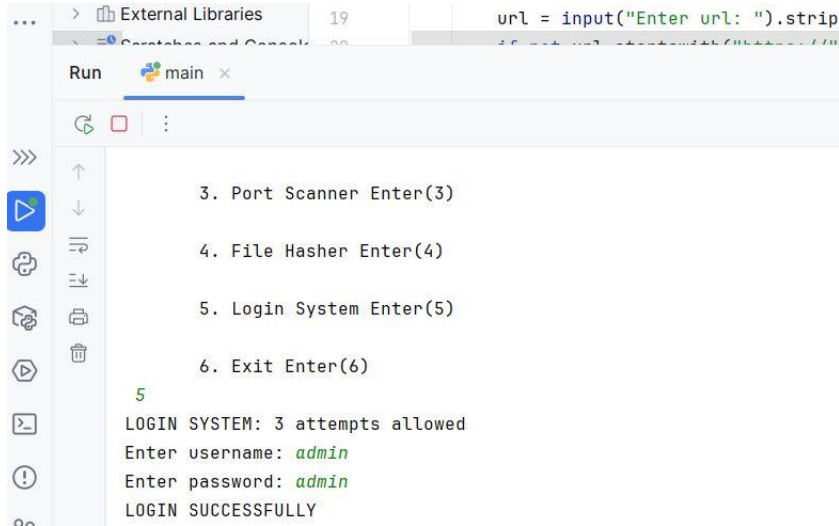
```

>>>
6. Exit Enter(6)

5
LOGIN SYSTEM: 3 attempts allowed
Enter username: admin
Enter password: admi
Wrong Credential Try Again
Enter username: admi
Enter password: admi
Wrong Credential Try Again
Enter username: admi
Enter password: admi
Wrong Credential Try Again
User blocked

```

When Correct Passkey Enter:



```
url = input("Enter url: ").strip
if not url.startswith('http://')
...
Run main x
3. Port Scanner Enter(3)
4. File Hasher Enter(4)
5. Login System Enter(5)
6. Exit Enter(6)
5
LOGIN SYSTEM: 3 attempts allowed
Enter username: admin
Enter password: admin
LOGIN SUCCESSFULLY
```

These outputs demonstrate the functionality and correctness of each module.

9. CONCLUSION

This project provided practical exposure to Python programming and fundamental cybersecurity concepts. It helped in understanding how real-world security tools work, including URL analysis, password validation, network scanning, and data hashing.

Through this project, key skills such as problem-solving, logical thinking, and implementation of security mechanisms were developed. The project successfully demonstrates a basic yet functional cybersecurity toolkit that can be expanded further into a more advanced system.